ImpactGuard

Validate business outcomes after code changes

A checkout can return HTTP 200 while fewer customers complete a purchase. A banking service can report success while the bank has not settled a transfer. These failures require business evidence as well as application telemetry.

ImpactGuard is a working prototype for developers and business operators, available locally and as a public Cloud Run demo. It measures outcome changes, inspects candidate commits, separates regression hypotheses from other anomalies, and records a human review decision. It does not modify application code or repair transactions.

Who would pay for it?

An independent Shopify seller needs to know whether a release reduced completed orders or introduced unprofitable discounts. A payment team needs to find ledger credits that lack settlement evidence. The product reduces the work required to assemble an investigation; its value depends on accurate data, useful attribution, and fewer false alerts.

Product question	Evidence to inspect
Did a release reduce checkout completion?	Comparable session windows, payment segments, deployed commit timing, changed redirect logic.
Did a discount change make orders unprofitable?	Order amounts, combined discounts, configured costs, changed promotion logic.
Are recorded credits backed by settled transfers?	Bank state, application state, immutable postings, exact integer-cent totals.

Public demonstration: impactguard-182850660018.us-central1.run.app. Repository: github.com/vishnusreddy/impactguard. All current demonstration scenarios use explicitly labeled synthetic data.

How it differs from existing observability products

New Relic is not limited to runtime exceptions. It supports custom events, business attributes, queries, dashboards, and alerts [1]. Sentry also provides release context and suspect-commit workflows [2]. Business metrics and commit correlation alone are not defensible claims of novelty.

ImpactGuard proposes a narrower workflow: validate a business outcome, examine a candidate code change, inspect the financial implication, and decide whether the finding is a regression or another kind of issue. It can complement an observability platform. The comparison below describes product emphasis, not exclusive capabilities.

Area	Established tools	ImpactGuard prototype
Primary workflow	Broad application and infrastructure observability; configurable custom telemetry.	Business-outcome investigation with commerce and settlement examples.
Business measurements	Custom events and metrics can represent purchases and other outcomes [1].	Opinionated completion-rate, margin, cohort, and settlement checks.
Code context	Release and suspect-commit features already exist in products such as Sentry [2].	Read-only, multiple-repository import; bounded diff inspection and heuristic candidate ranking.
Financial validation	Can be implemented using custom data and queries.	Exact-cent settlement checks and disclosed revenue-exposure formulas.
AI	Do not assume competing products lack AI.	Optional Vertex semantic classification and evidence-based investigation. Configured locally with personal Vertex AI.
Decision and action	Capabilities vary by product and configuration.	Manual resolve, ignore, not-an-issue, and reopen. No automatic repair.

The business hypothesis is faster investigation for a small team without a dedicated observability engineer. This remains to be tested with real customers; the prototype does not establish commercial superiority.

Seller example: Luma Goods

Luma Goods sells through its own Shopify storefront. The demonstration assumes 120 checkout starts per day and an $82 average order value. These inputs are assumptions, not collected seller revenue.

After a synthetic checkout release, completion declines for Safari sessions using PayPal while other segments remain near baseline. ImpactGuard shows the affected segment, sample sizes, a candidate redirect-handling diff, and estimated daily exposure. The temporal association is a hypothesis, not proof of causation.

Investigation sequence

Step	Operator workflow
1. Collect	In Integrations, select a sample case and collect Shopify and GA4 data. Generated data is clearly labeled. Real connectors remain disconnected.
2. Measure	Compare baseline and recent outcome rates. Validate collection completeness and traffic mix before assigning a cause.
3. Inspect code	Connect frontend, checkout, and backend repositories under one project. Inspect bounded commit diffs. Supply deployment mapping before attributing a live issue.
4. Analyze	Request Vertex analysis when personal credentials are configured. The model classifies the change and proposes verification steps.
5. Review	Mark resolved, ignore, mark not an issue, or reopen. Review state changes preserve source measurements and ledger evidence.

Revenue exposure = (baseline completion rate - current completion rate) x daily checkout starts x average order value. This is a projection under an unchanged-volume assumption, not realized loss or recovered revenue.

The source collector and the synthetic dashboard are separate. The current version does not join Shopify customers to GA4 users, schedule ingestion, or prove that an imported commit was deployed. Those production capabilities require explicit data definitions and deployment events.

Banking validation and issue classification

The banking fixture contains 2,000 transfers. In 200 cases, the application marks success while the bank reports failed or pending. The fixture contains $46,998.09 in credits without confirmed settlement. Exact integer-cent arithmetic computes the mismatch; a language model does not calculate the ledger balance.

The investigation points to a candidate status-mapping change that treats most non-cancelled statuses as success. An operator can review the evidence and mark the finding resolved, but that action does not reverse credits, alter transaction statuses, or assert that settlement has occurred.

Classification	Meaning and evidence requirement
Business regression	An outcome change with release and changed-behavior evidence. Candidate causation still requires verification.
Traffic anomaly	Traffic volume or mix changed. A spike alone does not establish a code regression.
Security signal	Possible attack indicators, such as elevated blocked requests. This does not establish a confirmed compromise.
Operational incident	Service, fulfillment, or process behavior changed without sufficient code-regression evidence.
Data quality	Missing, delayed, duplicated, or inconsistent measurements can explain the apparent change.
Inconclusive	Insufficient evidence or missing deployed-commit mapping. Do not force a regression verdict.

Sample cases include a checkout decline, a traffic spike, possible attack signals, and a purchase-event collection gap. The delivery cohort fixture is an operational incident. Security signals must remain separate from regressions.

Where AI is used

AI should interpret evidence and help investigate causes. Numerical validation must remain reproducible. Increasing the number of model calls does not increase reliability; the product uses AI where semantic judgment adds value.

Stage	Implementation	Current activation
Business measurement	SQL aggregates, comparable-window rates, exact-cent ledger checks.	Deterministic; available locally.
Synthetic scenario screening	Statistical and domain checks create demo findings.	Deterministic; not replaced by a model.
Source anomaly classification	Vertex reads aggregate evidence and configurable checks; returns a typed issue category.	Enabled and verified locally with Gemini 3.5 Flash-Lite.
Cause investigation	Vertex summarizes evidence and proposes verification steps. Bounded redacted candidate diffs can provide context.	Enabled and verified locally with Gemini 3.5 Flash-Lite.
Candidate commit ranking	Transparent timing, service, diff, and segment heuristic.	Deterministic; imported commits require deployment mapping.
Remediation	No model or deterministic auto-fix path.	Manual review only.

Cost controls

Analysis runs on an operator request, not on dashboard polling or every event. Requests contain compact aggregate evidence rather than raw customers. Input is bounded; output is limited to 1,200 tokens. Identical requests are cached in memory. The default budget is 20 model attempts per server process. Invalid model output is rejected without changing evidence.

Live AI is verified in a dedicated personal project. A checkout investigation used 1,163 input tokens and 106 output tokens; the repeated request returned from the local cache. At the current global standard rates of $0.30 per million input tokens and $2.50 per million output tokens, this request is approximately $0.0006, excluding taxes and other services [5]. This is an estimate, not a billing invoice.

No Vertex request used the work account. The application loads only explicitly selected personal credentials. When configuration is absent, the interface reports deterministic analysis.

Integrations, security, and deployment readiness

One project can connect up to 10 GitHub repositories. Import fetches recent commit metadata and bounded patches, filters sensitive filenames, and redacts known credential patterns before storage. Public repositories need no token; private import uses an explicit read-only token. The application does not read ambient GitHub CLI authentication.

Shopify collection uses the Admin GraphQL API [3], a configured store domain, and a dedicated token. It reads up to 500 recent orders without requesting customer or card details. GA4 collection uses the Data API [4] with a dedicated credential file and read-only scope; it requests daily sessions, purchases, and revenue. Live account collection has not been validated with user accounts.

Credential boundaries

Credentials remain in local environment variables or dedicated files excluded from Git. They are not included in AI prompts, application API responses, or provider-error logs. Provider authentication necessarily sends credentials only to the intended provider. Pattern redaction is defense in depth; it cannot identify every possible secret format. Keep secrets out of repositories and analysis instructions.

Personal Vertex setup

Configure IMPACTGUARD_VERTEX_PROJECT, IMPACTGUARD_VERTEX_CREDENTIALS, and GEMINI_MODEL. The adapter loads only the explicitly selected credential file. It does not use ambient gcloud credentials or the existing work project. Install optional Python dependencies before enabling live analysis.

Readiness and next steps

The app is deployed on scale-to-zero Cloud Run with durable Cloud Storage checkpoints. The public preview uses isolated synthetic data and cached Gemini investigations. Owner access protects account connections, private data, review changes, and new paid AI requests. It is not a multi-user production service. Before production, add authentication, managed secrets, scheduled collection, consistent metric definitions, release-to-repository mappings, source freshness checks, retention controls, and integration tests against selected accounts.

Recommended pilot: one seller, one checkout journey, known releases, and daily comparison against order records. Evaluate false-positive rate, investigation time, and whether candidate causes are useful. Do not claim prevented losses from manual review decisions.

Demonstration guide and references

Open Integrations and collect both samples. Analyze the checkout decline: without release mapping it remains inconclusive. Select Traffic spike and collect again; analyze to show a traffic anomaly. Select Possible attack signals to show a security signal, then the collection gap to show data quality. These examples demonstrate classification boundaries without account access.

Open Codebases to attach multiple repositories. Explain that a connected repository is not proof of deployment. Run the checkout or banking demo scenario, open its investigation, inspect candidate diffs, and mark the issue ignored or resolved. Show that the source metrics and ledger mismatches remain unchanged.

For an AI demonstration, configure a personal Vertex project and explicit credentials, then select Analyze metrics or Run analysis. Show the provider, classification, verification step, and cache behavior. If credentials are absent, present the feature as implemented but unconnected.

Primary references

1. New Relic: custom events and business attributes
https://docs.newrelic.com/docs/data-apis/custom-data/custom-events/report-custom-event-data/

2. Sentry: suspect commits
https://sentry.io/changelog/2023-12-4-improvements-to-suspect-commits/

3. Shopify: orders query
https://shopify.dev/docs/api/admin-graphql/latest/queries/orders

4. Google Analytics: properties.runReport
https://developers.google.com/analytics/devguides/reporting/data/v1/rest/v1beta/properties/runReport

5. Google Cloud: model pricing
https://cloud.google.com/gemini-enterprise-agent-platform/generative-ai/pricing

Local setup and integration details are maintained in docs/connections-and-ai.md. The older demo video includes the former simulated-remediation flow; use this document and the current interface for the updated review-only demonstration.